

Q: 1



Flipkart Order Intelligence & Support Assistant

Flipkart's catalog team wants one connected system, not three unrelated scripts. Customer support agents currently have no quick way to see whether an incoming order is likely to be returned, no automated way to double-check which catalog category a product photo belongs to when a listing is miscategorised, and no single assistant a support agent can ask a policy question and get a grounded answer from instead of hunting through a wiki. You are building the first version of that system end to end: a return-risk model trained on Flipkart's own order history, a product-image categoriser trained with transfer learning, and one support agent that can call both of those trained artifacts as tools while also answering policy questions from a small grounded knowledge base.

This is a single, connected build, not three disconnected exercises. Part 1 trains and saves a return-risk model. Part 2 trains and saves an image classifier. Part 3 is the one user-facing piece of the whole project -- a LangGraph agent that loads *both* saved artifacts from Parts 1 and 2 as real, callable tools, on top of its own retrieval-augmented policy knowledge base. Nothing you build in Parts 1-2 is thrown away after grading; Part 3 is the reason they exist.

Every technique, library, and pattern used below is one you have already practiced hands-on across classical ML, deep learning, and GenAI/agentic AI. No new tool, dataset, or concept is introduced. Every dataset named below is either a deterministic script you run yourself (Part 1) or a free, keyless, publicly hosted benchmark dataset with a pinned exact source (Part 2). Every AI-API dependency defaults to a free, keyless, fully offline mock mode (Part 3) -- you will never need a paid account or a credit card to complete this project, and no part of your grade depends on using a live LLM.

Total marks: 100 · One deliverable: a single public GitHub repository containing all three Parts' code, saved model artifacts, and a README tying them together into one working support-assistant demo.



Submission Guidelines (read first)

- This project has **exactly one submission: a public GitHub repository link**. Parts 1, 2, and 3 all live inside that one repo -- there is no separate submission per Part, and no link type other than a GitHub repository URL will be graded.
- The repo's `README.md` must document, in one place: how to regenerate Part 1's dataset and model, how to run Part 2's training/evaluation, and how to run Part 3's agent in its default mock mode, plus at least one full example transcript of the agent in action.
- No images, screenshots, diagrams-to-upload, PDFs, slide decks, presentations, video, or audio are required or accepted anywhere in this project. Every artifact is a committed text/code/data file in the repo. Product images used *inside* Part 2's own training pipeline (from the dataset you download programmatically) are not a submission requirement placed on you -- you never manually upload or attach an image yourself.
- Free-tier / free tools are sufficient for every task in this brief:
 - scikit-learn, pandas, numpy** (Part 1) -- fully free, local, no account.
 - PyTorch + torchvision, or TensorFlow/Keras** (Part 2) -- fully free, local; the Fashion-MNIST dataset downloads automatically with no login and no API key.
 - sentence-transformers (for embeddings) + Faiss or ChromaDB (for the vector index)** (Part 3) -- fully free, local, no account, no API key.
 - The agent's language-model calls default to a deterministic MOCK_LLM mode** that needs zero API keys and zero network access, and this is exactly what your graded transcripts must be run against. An **optional** free-tier live-LLM extension (e.g. Groq's free tier, or any other genuinely free & keyless-registration provider) may be wired in behind an `USE_LIVE_LLM=1` environment flag for your own exploration, but it is never required and never scored -- the repository must still run and pass every acceptance criterion with `USE_LIVE_LLM` unset.
- Originality: your dataset-generation script's output, trained models, agent code, and written analysis must be your own work product for this specific brief.
- Git workflow.** The repository's overall commit history (checked once against the whole repo, not per Part) must show at least one feature branch created, committed to at least twice, and merged into `main` (visible via `git log --graph --all` or the repository's branch/merge-commit history).
- Submit your one repository link by the end of Day 14 of the project window.

Part 1 -- Return-Risk Scoring Pipeline (35 marks)

Flipkart's category-management team wants to flag orders that are statistically likely to be returned *before* the return happens, so a support agent (and, later, your own Part 3 agent) can proactively check an order's risk. Everything you build here is the fixed input for Part 3 -- you may not swap in a different dataset or a different final model once this Part is graded.

Tasks

1. **Generate the exact seeded dataset.** Save the script below as `generate_orders.py` in your repo and run it exactly as written (`python3 generate_orders.py`) -- do not change `np.random.default_rng(42)` or any of the fixed category/payment lists, since the acceptance criteria below depend on this exact, deterministic output.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
N = 6000

categories = ["Apparel", "Electronics", "Home", "Footwear", "Beauty"]
cat_probs = [0.32, 0.22, 0.18, 0.18, 0.10]
payment_methods = ["COD", "Prepaid_Card", "Prepaid_UPI", "Wallet"]
pay_probs = [0.42, 0.24, 0.24, 0.10]

product_category = rng.choice(categories, size=N, p=cat_probs)
payment_method = rng.choice(payment_methods, size=N, p=pay_probs)

base_price = {
    "Apparel": (400, 2200), "Electronics": (1200, 45000), "Home": (300, 8000),
    "Footwear": (500, 4500), "Beauty": (150, 2500),
}
price_inr = np.round(np.array([rng.uniform(*base_price[c]) for c in product_category]), 0)

discount_pct = np.clip(rng.normal(22, 15, N), 0, 75)
customer_tenure_days = np.clip(rng.exponential(380, N), 1, 2500).round(0)
num_previous_orders = np.clip((customer_tenure_days / 45) + rng.normal(0, 2, N), 0, None).round(0)
base_return_rate = np.clip(rng.beta(1.5, 9, N), 0, 1)
num_previous_returns = np.round(base_return_rate * num_previous_orders).clip(0, num_previous_orders)

delivery_distance_km = np.clip(rng.gamma(3, 90, N), 2, 2200).round(1)
delivery_days = np.clip(rng.normal(4.5, 2.2, N), 1, 21).round(0)
is_weekend_order = rng.integers(0, 2, N)

rating_given = rng.integers(1, 6, N).astype(float)
missing_mask = rng.random(N) < np.where(payment_method == "COD", 0.22, 0.06)
rating_given[missing_mask] = np.nan

fit_risk_cat = np.isin(product_category, ["Apparel", "Footwear"]).astype(float)
prev_return_ratio = np.where(num_previous_orders > 0,
                              num_previous_returns / np.maximum(num_previous_orders, 1), 0)

z = (-2.2 + 1.9 * prev_return_ratio + 0.55 * fit_risk_cat
      + 0.014 * (discount_pct - 20) / 10 + 0.9 * (payment_method == "COD").astype(float)
      + 0.10 * (delivery_days - 4.5) / 2 + 0.30 * (price_inr / base_price["Electronics"][1])
      + 0.05 * is_weekend_order - 0.15 * np.tanh(customer_tenure_days / 500))
```

```

prob_return = 1 / (1 + np.exp(-z))
returned = (rng.random(N) < prob_return).astype(int)

df = pd.DataFrame({
    "order_id": np.arange(1, N + 1), "product_category": product_category,
    "price_inr": price_inr, "discount_pct": np.round(discount_pct, 1),
    "payment_method": payment_method, "customer_tenure_days": customer_tenure_days.astype(int),
    "num_previous_orders": num_previous_orders.astype(int),
    "num_previous_returns": num_previous_returns.astype(int),
    "delivery_distance_km": delivery_distance_km, "delivery_days": delivery_days.astype(int),
    "is_weekend_order": is_weekend_order, "rating_given": rating_given, "returned": returned,
})
df.to_csv("orders_dataset.csv", index=False)
print("Rows:", len(df), "| Return rate:", round(df["returned"].mean(), 4))

```

Commit both `generate_orders.py` and the resulting `orders_dataset.csv` to your repo.

2. **Verify the generated data.** Report: total row count, overall return rate, percentage of missing `rating_given` values, and a table of return rate broken out by `product_category` and separately by `payment_method`. State explicitly whether the missingness pattern in `rating_given` looks like MCAR, MAR, or MNAR, and justify your answer from how the column was actually generated (hint: its missingness depends on another observed column).
3. **Preprocess without leakage.** Using a `ColumnTransformer` + `Pipeline` (scikit-learn), impute missing numeric values with the median and missing categorical values with the mode, one-hot encode `product_category` and `payment_method`, and standard-scale the numeric features. Fit the pipeline on the training split only, then transform both splits -- never fit on the test split.
4. **Build a baseline.** Train a `DummyClassifier` (most-frequent strategy) on a stratified 80/20 train/test split (`random_state=42`). Report its accuracy and its F1-score for the `returned=1` class, and write one paragraph explaining, in plain language, why a high accuracy number here is misleading -- name the specific failure mode (comparing to a baseline, and metrics aligned to the real business problem, are two of the five honest-evaluation rules this task is built on).
5. **Train and tune a Logistic Regression model.** Use `class_weight="balanced"` to address the imbalance. At the default 0.5 threshold, report accuracy, F1, recall, precision, and ROC-AUC for the `returned=1` class. Then sweep the decision threshold from 0.1 to 0.9 in steps of at most 0.02, plot (or tabulate) F1 against threshold, and report the threshold that maximises F1 along with its recall/precision at that threshold. Write one paragraph stating the business trade-off this threshold change represents (which kind of error gets more expensive to avoid, and which kind you are accepting more of).
6. **Train and tune a Random Forest model.** Wrap the same preprocessing pipeline with a `RandomForestClassifier(class_weight="balanced", random_state=42)` and run `GridSearchCV` over at least `n_estimators` in `[100, 200]` and `max_depth` in `[6, 10, None]`, scored on `roc_auc` with 5-fold `StratifiedKFold` cross-validation. Report the best parameter combination, the best cross-validated ROC-AUC, and the held-out test-set ROC-AUC for the winning configuration.
7. **Explain the model.** Extract feature importances from the winning Random Forest (via `.feature_importances_`, or via SHAP if you install the `shap` package -- both are free, local libraries). Report the top 5 most important features and write one paragraph interpreting why each one plausibly drives return risk for an e-commerce order. **Then compute `sklearn.inspection.permutation_importance` on the held-out test split for the same top-5 features** and compare the two rankings side by side. Impurity-based `.feature_importances_` is biased toward high-cardinality continuous columns regardless of whether they actually carry signal; permutation importance (which measures the real drop in test-set performance when a feature is shuffled) does not share this bias. Name explicitly which of your original top-5 features lose most of their importance under the permutation measure, and explain in one sentence why impurity-based importance can overrate a noisy continuous feature.

8. **Subgroup / root-cause analysis.** Break out the winning model's recall and precision by `product_category` and separately by `payment_method` on the test set. Name at least one subgroup where the model performs meaningfully

worse than its overall average, and propose one concrete, specific next step to address it (e.g. a category-specific threshold, an added feature) -- do not just say "collect more data."

9. **Save the artifact.** Your final chosen pipeline is the **tuned Random Forest from Task 6** (preprocessing + the `GridSearchCV` -selected model together, as one fitted scikit-learn `Pipeline`) -- not the Logistic Regression, which exists in this project only as the simpler baseline Task 5 sweeps a threshold over. Persist it with `joblib.dump(...)` to `models/return_risk_model.pkl` . This file is what Part 3's `check_return_risk` tool will load. **Before saving, re-run Task 5's threshold-sweep procedure one more time, but this time on the Random Forest's own `predict_proba` output on the test split** (not the Logistic Regression's), and record the resulting F1-maximising threshold as `t*_rf` . This is the value Part 3's tool will anchor its risk buckets to -- reusing the Logistic Regression's `t*` from Task 5 would be calibrating one model's bucket cut points using a different model's probability scale, which is not guaranteed to produce a sensible split.

Acceptance criteria

- `orders_dataset.csv` has exactly 6,000 rows and 13 columns as specified by the generator above.
- The reported overall return rate is between 18% and 27%, and `rating_given` is missing on between 8% and 18% of rows (both are properties of the exact seeded generator, not something you need to tune).
- The missingness classification names **MAR** (missing-at-random, conditional on the observed `payment_method` column) and explicitly states the measured missing-rate gap between `COD` and non-`COD` orders as the evidence -- not MCAR (there is a real dependency) and not MNAR (the missingness does not depend on the unobserved `rating_given` value itself).
- The baseline `DummyClassifier` 's F1-score for `returned=1` is reported as `0.0` (or documented as such), and the accompanying paragraph correctly names "high accuracy, zero recall" as the trap.
- The default-threshold Logistic Regression achieves ROC-AUC of at least 0.58 and F1 (class 1) of at least 0.30 on the held-out test set.
- The threshold sweep produces a chosen threshold whose recall for class 1 is at least 15 percentage points higher than the default-threshold recall, with the corresponding precision drop stated numerically.
- The Random Forest `GridSearchCV` reports a best cross-validated ROC-AUC of at least 0.58, and the test-set ROC-AUC is within 0.05 of that cross-validated value (evidence against severe overfitting).
- The top-5 `.feature_importances_` list includes `payment_method` (in one-hot form) and at least two of `price_inr` , `customer_tenure_days` , `discount_pct` , `num_previous_returns` -- `delivery_distance_km` is deliberately excluded from this required group (it is not a feature the return-generating process actually depends on; see Task 7's permutation-importance step). The permutation-importance comparison from Task 7 is also present, correctly identifies at least one original top-5 feature whose importance drops substantially under permutation, and states in one sentence why impurity-based importance can overrate a noisy continuous column.
- The subgroup table is present for both `product_category` and `payment_method` , with correct arithmetic, and names a genuinely weaker subgroup with a specific (not generic) proposed fix.
- `models/return_risk_model.pkl` exists, loads without error via `joblib.load` , is the tuned Random Forest pipeline from Task 6 (not the Logistic Regression), and its `.predict_proba` output is what Part 3's tool actually calls (not a hardcoded stand-in). `t*_rf` is reported -- the F1-maximising threshold computed on this saved model's own `predict_proba` on the test split (re-running Task 5's sweep procedure on this model, not reusing the Logistic Regression's threshold).
- The repository's overall commit history shows a feature branch created, committed to at least twice, and merged into `main` -- checked once across the whole repository, not per Part.

🔗 Part 2 -- Product Image Categoriser via Transfer Learning (25 marks)

Flipkart's catalog team occasionally gets a product photo with no category tag, or a suspiciously mismatched one. Using **the standard transfer-learning approach for CNN-based image classification** -- a pretrained backbone with early/middle layers frozen and only a new classifier head (and, if needed, some late layers) retrained -- build a categoriser for exactly the kind of apparel/footwear/accessory catalog items Flipkart sells. This Part does not depend on Part 1's data, but its saved model is consumed by Part 3 exactly the way Part 1's is.

Tasks

- 1. Load the dataset.** Use **Fashion-MNIST** (Zalando Research), the free, keyless, automatically-downloadable benchmark of 70,000 grayscale 28x28 product images across 10 categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot -- an exact match for Flipkart's own apparel/footwear/accessories catalog. Pinned source: <https://github.com/zalandoresearch/fashion-mnist> (also fetchable with zero configuration via `torchvision.datasets.FashionMNIST(root=..., download=True)` , which pulls from this same canonical dataset). Use the standard 60,000-image train split and 10,000-image test split; carve a stratified validation split (at least 5,000 images) out of the training set for model selection, leaving the test split untouched until final evaluation.
- 2. Preprocess for a pretrained backbone.** Replicate the single grayscale channel to 3 channels, resize to whatever input size your chosen pretrained backbone expects (document the exact size you use), and normalize with the ImageNet mean/std the backbone was originally trained with.
- 3. Build the transfer-learning model.** Load a pretrained CNN backbone (ResNet-18 or EfficientNet-B0), freeze the early and middle layers, and attach a new classifier head sized for 10 output classes. Train only the new head first (feature extraction). Document your batch size, optimizer (use Adam), learning rate, and number of epochs. **Speed tip (do this, it changes the runtime by roughly an order of magnitude):** since the backbone is frozen during feature extraction, run it once over every image to extract and cache its output features, then train only the small head on those cached feature vectors -- this is mathematically identical to re-running the frozen backbone's forward pass every epoch, but turns an hours-long CPU training loop into one dominated by a single feature-extraction pass (a few minutes on a GPU, and still well under an hour on a laptop CPU) followed by a near-instant head-only training step. If you have access to a free GPU runtime (e.g. Google Colab or Kaggle), that is also a fine alternative to caching, but caching alone is sufficient on a CPU-only machine.
- 4. Fine-tune if needed.** If your feature-extraction validation accuracy is below 80%, unfreeze the backbone's late layers (keep early/middle layers frozen) and continue training at a lower learning rate, using the standard gradual-unfreezing fine-tuning strategy. Document the before/after validation accuracy either way.
- 5. Evaluate.** Report final test-set accuracy, a full 10x10 confusion matrix, and per-class precision/recall.
- 6. Document confusion patterns.** Name at least two specific category pairs your model confuses most often (read them directly off your own confusion matrix -- do not guess), and write one paragraph per pair explaining, in terms of the actual visual similarity between those two apparel/footwear silhouettes, why the confusion is plausible.
- 7. Save the artifact.** Persist the trained model's weights (`torch.save(model.state_dict(), ...)` or the Keras/TensorFlow equivalent) to `models/product_classifier.pt` , plus a short, documented one-function loading + single-image-prediction snippet. This is what Part 3's `classify_product_image` tool will call.
- 8. Export real sample images as actual image files.** `torchvision.datasets.FashionMNIST` stores its data as raw IDX binary files, not a folder of individual images -- there is no ready-made image file for Part 3's `classify_product_image(image_path: str)` tool to point at. Pick at least 5 real test-split images (ideally covering different classes) and write each one out as an actual `.png` file via `PIL.Image.fromarray(...)` (or the equivalent in your chosen framework) to `data/sample_images/` , named so the true label is obvious from the filename (e.g. `data/sample_images/03_sneaker.png`). Commit these exact `.png` files to your repo -- this is what Part 3's tool will be pointed at, not the raw IDX data.

Acceptance criteria

- Uses Fashion-MNIST from the pinned source exactly -- no substitute dataset.
- Reports the exact train / validation / test split sizes actually used, with the test split untouched until the final reported number.
- States explicitly whether feature extraction alone was sufficient or fine-tuning was required, with the before/after validation accuracy numbers either way.
- Achieves at least 80% accuracy on the held-out test split. If, after attempting fine-tuning, this bar is genuinely not reached, the honest reported shortfall together with the attempted fine-tuning step and a written diagnosis of the confusion matrix is what is graded here -- never a fabricated number.
- The confusion matrix is generated from real model predictions (not simulated), and at least two real, visually-plausible category-confusion pairs are named with a specific explanation.
- `models/product_classifier.pt` (or equivalent) exists and is loadable by the documented snippet, and that snippet is what Part 3's tool actually calls.
- `data/sample_images/` contains at least 5 real, actual `.png` files exported from the test split (not the raw IDX data), and Part 3's `classify_product_image` tool is pointed at these exact files.

Part 3 -- Flipkart Support Agent (40 marks)

This is the one user-facing piece of the whole project. Build a single LangGraph agent that a Flipkart support workflow could actually call: it answers policy questions grounded in a small knowledge base you write, and it can call two real tools -- **Part 1's saved return-risk model** and **Part 2's saved image classifier** -- to answer order-risk and product-category questions. Nothing here may be a hardcoded stand-in for either earlier Part's model; the tools must load and call the real saved artifacts.

Tasks

1. **Write the policy knowledge base.** Author at least 12 short (2-4 sentence) Flipkart-style policy documents covering, at minimum: return windows by category (apparel/footwear vs. electronics vs. home), COD refund timelines, delivery SLAs, and reverse-pickup eligibility. Chunk them sentence-wise (one effective strategy for production RAG, over the fixed-size or overlapping alternatives -- multi-sentence documents will each produce more than one chunk). For at least 5 realistic test queries, record which one or two **documents** (not individual chunks) you would consider "relevant" -- this becomes your retrieval-evaluation answer key in Task 10. Keep a mapping from every chunk back to its parent document, since Task 10's scoring is done at the document level.
2. **Embed and index.** Embed every chunk with a free, local sentence-transformer model (e.g. `all-MiniLM-L6-v2`) and build a vector index over them with **Faiss** or **ChromaDB**. Both are free, local, and require no account or API key.
3. **Implement `check_return_risk(order_features: dict) -> dict`.** This tool loads Part 1's `models/return_risk_model.pkl` (the tuned Random Forest) and returns the model's predicted return probability plus a risk bucket. **Define the bucket cut points relative to `t*_rf` from Part 1 Task 9 (the F1-maximising threshold computed on the Random Forest's *own* `predict_proba`, not the Logistic Regression's), not fixed values** -- e.g. "Low" if probability < `t*_rf`, "High" if probability >= `t*_rf` + 0.15, else "Medium". Fixed cut points like 0.3/0.6 are not self-calibrating: two students with equally valid Random Forest models can get probability distributions concentrated in completely different ranges (one model's predictions might cluster mostly in the 0.3-0.6 band regardless of true risk, while a different valid hyperparameter choice spreads them much wider), so a fixed 0.3/0.6 split can silently collapse almost every real order into one bucket. Anchoring to your own `t*_rf` keeps the buckets meaningful for whatever probability range your specific trained Random Forest actually produces. State your resulting cut points and `t*_rf` value in one sentence.
4. **Implement `classify_product_image(image_path: str) -> dict`.** This tool loads Part 2's saved classifier and returns the predicted category label plus the model's confidence for that prediction, run against the real `.png` files you exported to `data/sample_images/` in Part 2 Task 8 (no new image upload is required or expected from you or from any grader -- the tool reads the files already committed to your repo).
5. **Build the LangGraph graph.** Construct a graph with at least 4 nodes, including: an intent node (decides whether the user is asking a policy question, a return-risk question, or a product-category question), a RAG-retrieval node, a tool-calling node, and a response-generation node. Use at least one **conditional edge** so the graph actually branches by intent rather than always running every node in sequence. Maintain short-term conversational state across at least one multi-turn exchange (e.g. a follow-up question that refers back to an order ID mentioned two turns earlier), **and show this working in your README transcript. Also show the same conversational-state mechanism in a freshly-started conversation** (a new graph invocation with no prior turns) correctly starting with that state absent/reset -- **include both the multi-turn transcript and the fresh-conversation transcript in your README so a grader can see state carried in one case and correctly cleared in the other.**
6. **Prompt-engineer the response generator.** Build the system prompt using the 4S principles (Specific, Short, Surround, Single) plus role prompting (e.g. "You are Flipkart's support assistant..."), include at least 2 few-shot examples for the intent-classification step, and require the final answer to come back in a fixed, structured JSON schema (fields: `answer`, `source` -- one of "policy_kb", "return_risk_tool", "image_classifier_tool" -- and `confidence`).
7. **Implement `MOCK_LLM` deterministic mode.** Write a rule-based/template function that, given the retrieved KB chunk(s) and/or tool output, deterministically composes the final structured answer with zero network calls and zero API keys. This is the default mode and is what every graded transcript in Task 9 must be run against.
8. **Add guardrails.** Implement input-side prompt-injection filtering (block or flag inputs containing patterns like "ignore previous instructions", "ignore all rules", "pretend you are..."), and an output-side groundedness check that refuses to answer a policy question if no retrieved chunk clears a minimum similarity threshold (rather than letting the mock generator fabricate a policy that isn't in the knowledge base).
9. **Run and record 8+ test conversations.** Cover, at minimum: (a) two different policy questions answered via RAG, (b) one return-risk question that calls `check_return_risk` with realistic order features, (c) one product-category question that calls `classify_product_image` against one of the real `.png` files in `data/sample_images/`, (d) one

multi-turn exchange demonstrating state carried across turns plus the matching fresh-conversation transcript from Task 5 showing state correctly absent, (e) one deliberate prompt-injection attempt that must be visibly blocked or deflected by your Task 8 input-side guardrail, and (f) one policy question with no sufficiently-similar retrieved chunk, where the agent's output-side groundedness check from Task 8 correctly refuses to answer rather than fabricating a policy -- print the retrieved chunk's similarity score and your threshold in this transcript so the refusal is verifiable. Save the full transcripts in `transcripts/` and link them from the README.

10. **Evaluate retrieval.** Using the 5+ query/relevant-document pairs from Task 1, compute `Precision@3` and `Recall@3` for each query and report the average of each across all queries, showing the per-query arithmetic (not just the final averages).

Acceptance criteria

- The knowledge base has at least 12 chunked documents; embeddings are computed with a free local model; the vector index is built with Faiss or ChromaDB (not a paid or account-gated vector service).
- Both tools are real function calls that load Part 1's and Part 2's actual saved artifacts -- spot-checking one tool call against directly running the saved model outside the agent produces the same number. `classify_product_image` is pointed at real `.png` files inside `data/sample_images/` (from Part 2 Task 8), not at raw IDX data or a hardcoded label.
- `check_return_risk`'s risk-bucket cut points are anchored to `t*_rf` -- the F1-maximising threshold computed on the saved Random Forest's *own* `predict_proba` output (not a fixed 0.3/0.6 split, and not the Logistic Regression's threshold from Task 5) -- and the one-sentence justification correctly states both the cut points used and the `t*_rf` value they are anchored to.
- The LangGraph graph has at least 4 nodes and at least one conditional edge; the README's multi-turn transcript demonstrably shows state (not memory) carried within one conversation, and a **separate** fresh-conversation transcript shows that same state correctly absent/reset.
- The system prompt is annotated against each of the 4S principles (Specific, Short, Surround, Single) plus role prompting, and at least 2 few-shot intent-classification examples are shown actually driving correct intent routing on at least 2 of the required transcripts (not merely present in the prompt text with no visible effect).
- `MOCK_LLM` mode requires zero API keys and zero outbound network calls, and all 8+ Task 9 transcripts are run in this mode (an optional live-LLM run may additionally be included but is never a substitute for the mock-mode transcripts).
- At least one of the 8+ transcripts is the prompt-injection attempt from Task 9(e), and it is visibly deflected -- the agent does not comply with the injected instruction.
- At least one of the 8+ transcripts is the ungrounded-question attempt from Task 9(f), and the agent's output-side groundedness check visibly refuses to answer, printing the retrieved chunk's similarity score against the stated threshold rather than fabricating a policy answer.
- `Precision@3` and `Recall@3` are computed at the **document** level (each retrieved chunk mapped back to its parent document and deduplicated before scoring against Task 1's document-level answer key) for at least 5 queries, with visible per-query arithmetic, not just final averages.
- If the optional live-LLM extension (Task-independent, entirely optional) is included, the README clearly marks it optional, and removing the API key still leaves every other acceptance criterion satisfied via `MOCK_LLM`.

Submission

Submit **one link**: your public GitHub repository. The repository must contain, at minimum: `generate_orders.py` and `orders_dataset.csv` plus your Part 1 training/evaluation code and `models/return_risk_model.pkl` (Part 1); your Part 2 training/evaluation code, confusion-matrix output, and `models/product_classifier.pt` (Part 2); your Part 3 knowledge-base files, vector-index build code, both tool implementations, the LangGraph agent code, `transcripts/` with all 8+ test conversations, and your retrieval-evaluation numbers (Part 3); and one root `README.md` that ties all three Parts together with clear run instructions for each. Do not submit separate links or files per Part, and do not attach any image, screenshot, slide deck, video, or audio file, or a PDF/document export of anything -- every artifact beyond the one GitHub repository link is a committed text/code/data file inside that one repository.